

Access to everything. Now up to 60% off. [Upgrade now](#)

Towards Dev · [Follow publication](#)

C++26 #embed: Finally, a Standard Way to Embed Files into Your Program

A complete guide to the new C++26 `#embed` preprocessor directive, its optional parameters, and how to use `__has_embed`

13 min read · 5 hours ago



Sagar

Following ▾



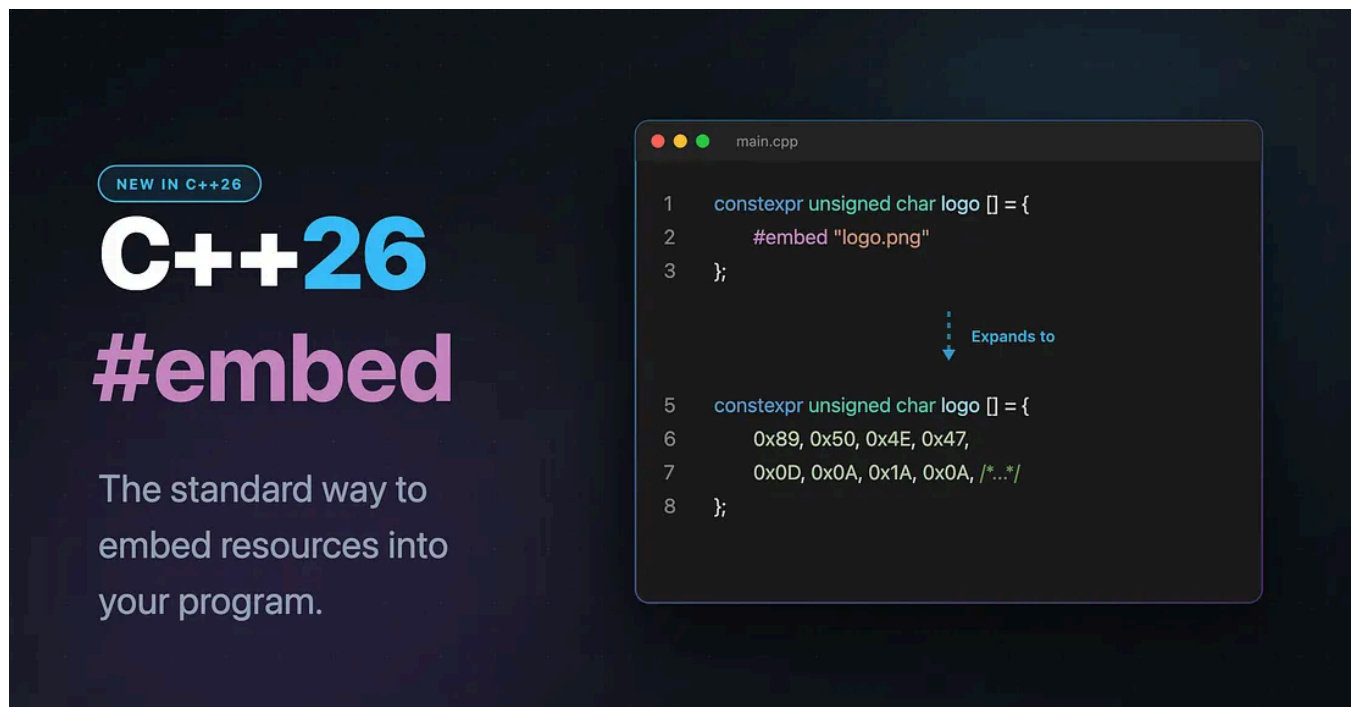
Listen



Share



More



For years, C++ developers have had one annoying problem that never had a standard solution. *What if your application needs a file at runtime?*

Many of us have solved this problem dozens of different ways over the years. The surprising part is that C++ **itself never provided a standard solution.**

That finally changes in C++26 with a brand new preprocessor directive `#embed` .

Instead of manually converting files into C++ arrays or relying on platform-specific tools, the compiler can now embed the contents of a file directly into your program. It is a small feature, but one that removes a surprising amount of build complexity.

For readers interested in the proposal behind this feature, the official C++26 paper is available here: [P1967R14 — #embed: A Standard Way to Embed Resources in C++](#).

. . .

The Problem: The Challenge of Bundling Resources

Imagine you're writing a game. The game needs a shader.

```
assets/  
  lighting.spv
```

At runtime, you might load it like this:

```
std::ifstream shader("assets/lighting.spv", std::ios::binary);
```

This is perfectly reasonable. Until one day someone reports:

"The game crashes immediately after installation."

You investigate. The executable is there. The shader isn't. The installer forgot to include it. Now imagine an embedded system. Instead of a shader, you need firmware.

```
firmware.bin
```

Or perhaps you're writing a networking library that ships with a default SSL certificate. Or maybe your application has a small icon that never changes.

These resources are really part of your program. They aren't user data. They're not configuration files. They're part of the application itself. So why should they live outside the executable?

For years, there wasn't a portable answer to that question. But we are engineers, one way or another we find a solution. Lets look at those.

. . .

How We Solved This Before C++26?

Since C++ offered no standard solution, developers came up with their own. Some approaches were simple. Some were clever. Almost all had drawbacks.

Let's look at the most common ones.

Option 1 — Load the File at Runtime

This is probably the most common solution.

```
std::ifstream file("logo.png", std::ios::binary);
```

It works well. But it introduces a new problem. Your executable now depends on the file existing at exactly the expected location. If someone deletes the file Or renames it Or packages your application incorrectly, Your program breaks.

Deployment becomes more complicated because you now have multiple files to distribute instead of just one executable.

Option 2 — Convert the File into a C++ Array

Many developers use tools such as `xxd` to generate C++ code from a binary file. For example `logo.png` becomes something like:

```
unsigned char logo[] = {  
    0x89, 0x50, 0x4E, 0x47,
```

```
    0x0D, 0x0A, 0x1A, 0x0A,  
    ...  
};
```

Now the file is compiled directly into your executable.

Problem solved? Not quite.

Someone now has to regenerate this file every time `logo.png` changes.

That usually means adding another build step. It also creates huge generated source files that nobody actually wants to read or review.

Option 3 — Linker or Platform-Specific Tricks

Many platforms provide ways to bundle binary resources into an executable.

Depending on the operating system or compiler, you might use:

- linker scripts
- resource compilers
- object file conversion tools
- platform-specific APIs

These approaches often work well. The downside is portability. A solution that works perfectly on one platform may require an entirely different build process on another. If your project targets multiple operating systems, things become even more complicated.

None of These Were Standard!

This was the real issue. Every team solved the problem differently. One project generated C++ headers. Another relied on linker scripts. Another used custom build tools. Another simply loaded files at runtime and hoped they were present. The C++ standard had no opinion on how this should be done.

That meant every build system, every project, and every company invented its own solution.

C++26 changes that by introducing a portable, standard way to embed resources directly into your program using a single directive: `#embed`

No generated headers. No custom conversion tools. No linker-specific tricks. Just standard C++.

• • •

Why `#embed` Is a Preprocessor Directive?

One question you might have is:

Why isn't this just another library feature?

Because the compiler needs access to the file **before compilation begins**.

When the preprocessor encounters: `#embed "logo.png"` it reads the contents of the file and replaces the directive with the file's bytes as a comma-separated sequence of integer values.

From the compiler's point of view, it's as if you had written those values yourself in the source code. This preprocessing behavior is why `#embed` is implemented as a new preprocessor directive rather than a standard library function.

Ok enough warm-up time to use it.

• • •

Using `#embed`

Now that we understand the problem `#embed` solves, let's see how it actually works.

The good news is that the syntax is surprisingly simple.

At its most basic, you simply provide the path to a file.

```
constexpr unsigned char logo[] = {  
    #embed "logo.png"  
};
```

That's it. There is no conversion tool. No generated header. No custom build step. When the compiler processes this file, it reads `logo.png` during preprocessing and replaces the `#embed` directive with the contents of the file.

Conceptually, it becomes something like this:

```
constexpr unsigned char logo[] = {  
    0x89, 0x50, 0x4E, 0x47,  
    0x0D, 0x0A, 0x1A, 0x0A,  
    ...  
};
```

You never see this generated code — the compiler does it for you — but that's effectively what happens. The result is exactly what many of us have been generating manually for years. Except now it's part of the language.

. . .

What Does `#embed` Actually Produce?

One common misconception is that `#embed` creates an array.

It doesn't.

It simply expands into a comma-separated list of integer literals representing the bytes of the file.

For example, suppose we have a tiny file named `hello.txt` containing:

```
Hi
```

The ASCII values are:

```
'H' -> 72  
'i' -> 105
```

So this `#embed "hello.txt"` conceptually expands to `72,105` .

Notice that there's no array. No variable. Just a sequence of integers.

That's why `#embed` can only be used where a comma-separated list of values makes sense.

For example:

```
constexpr unsigned char text[] = {  
    #embed "hello.txt"  
};
```

or

```
std::array<unsigned char, 2> data{  
    #embed "hello.txt"  
};
```

Both are perfectly valid because the embedded bytes become initializer elements.

. . .

Embedding Binary Files

The best part is that `#embed` isn't limited to text files.

It works just as well with binary files.

For example all the below just works fine as it is:

```
constexpr unsigned char firmware[] = {  
    #embed "firmware.bin"  
};  
  
constexpr unsigned char shader[] = {  
    #embed "lighting.spv"  
};  
  
constexpr unsigned char model[] = {
```

```
#embed "model.bin"  
};
```

The compiler doesn't care what the file contains. It simply reads the raw bytes and inserts them into your program. Whether it's an image, firmware, compressed archive, or machine learning model, the process is exactly the same.

• • •

Finding the File

`#embed` accepts two familiar forms for specifying a resource.

Using double quotes:

```
#embed "logo.png"
```

Or using angle brackets:

```
#embed <logo.png>
```

Just like `#include`, the compiler searches for the resource according to its implementation-defined search rules. In practice,

- **quoted paths** are typically used for project resources,
- while **angle brackets** are intended for resources found through configured search paths.

The exact lookup behavior depends on your compiler.
For most projects, you'll probably use the quoted form.


```
#embed "assets/logo.png"
```

It feels natural and matches how most projects organize their resources.

• • •

Optional Parameters

So far, we've used `#embed` in its simplest form.

```
constexpr unsigned char logo[] = {  
    #embed "logo.png"  
};
```

For many applications, that's all you'll ever need.

But C++26 also provides a few optional parameters that give you more control over how the embedded data is generated.

These parameters let you:

- embed only part of a file
- add values before or after the embedded bytes
- handle empty files gracefully

Let's look at each one.

• • •

`limit` — Embed Only the First N Bytes

Sometimes you don't need the entire file. Maybe you're only interested in:

- checking a file signature
- reading a header

- validating a magic number
- embedding a small preview

Instead of embedding the entire resource, you can limit how many bytes are included.

```
constexpr unsigned char header[] = {  
    #embed "firmware.bin" limit(16)  
};
```

Only the first **16 bytes** are embedded.

If the file is larger, everything after those 16 bytes is ignored.

If the file contains fewer than 16 bytes, all available bytes are embedded.

This can be useful when inspecting binary formats.

For example, many image formats begin with a unique signature.

PNG files start with:

```
89 50 4E 47 0D 0A 1A 0A
```

Instead of embedding the entire image, you can embed just enough bytes to verify its format.

```
constexpr unsigned char signature[] = {  
    #embed "logo.png" limit(8)  
};
```

This keeps the generated data small while still giving you the information you need.

• • •

prefix — Insert Values Before the Embedded Data

Sometimes you want additional values to appear before the file contents. That's exactly what `prefix` does.

Suppose the file contains:

```
10 20 30
```

Using:

```
constexpr unsigned char data[] = {  
    #embed "data.bin" prefix(1, 2, 3,)  
};
```

Conceptually produces:

```
constexpr unsigned char data[] = {  
    1, 2, 3,  
    10, 20, 30  
};
```

Notice the **comma** after `3`. This often looks a little unusual at first.

The reason is that `prefix()` accepts a sequence of preprocessing tokens, allowing them to blend naturally with the embedded bytes without the compiler needing to insert commas automatically.

A practical example might be adding a version number before some embedded firmware.

```
constexpr unsigned char firmware[] = {  
    #embed "firmware.bin"  
    prefix(1,)  
};
```

Now the first byte represents the firmware version, followed by the actual firmware image.

. . .

`suffix` — Insert Values After the Embedded Data

As you might expect, `suffix` works in the opposite direction. Suppose the file contains:

```
10 20 30
```

This:

```
constexpr unsigned char data[] = {  
    #embed "data.bin" suffix(40, 50,  
};
```

conceptually becomes:

```
constexpr unsigned char data[] = {  
    10, 20, 30,  
    40, 50  
};
```

One common use is adding a terminating null character.

```
constexpr char certificate[] = {  
    #embed "certificate.pem"  
    suffix('\0')  
};
```

Now the embedded data can be treated as a C-style string. Without the suffix, the embedded bytes would not automatically include a null terminator.

• • •

Combining `prefix` and `suffix`

You can use both together.

```
constexpr unsigned char packet[] = {  
    #embed "payload.bin"  
    prefix(0xAA,)  
    suffix(0x55,)  
};
```

If the file contains:

```
10 20 30
```

The final result is conceptually:

```
0xAA,  
10,  
20,  
30,  
0x55
```

This can be useful when constructing binary protocols where packets require fixed header and trailer bytes.

• • •

`if_empty` — Handling Empty Files

What happens if the file exists but contains no data?

Normally, `#embed` simply contributes no values. Sometimes that's exactly what you want. Other times, you'd rather insert a default value instead.

That's where `if_empty` comes in.

```
constexpr unsigned char data[] = {  
    #embed "config.bin"  
    if_empty(0)  
};
```

If `config.bin` contains data, those bytes are embedded as usual.

If the file is empty, the value `0` is inserted instead.



If file contains: `10 20 30` it will result in `10,20,30` .

If the file is empty, it will result in `0` .

This can be useful when you always expect an initializer list to contain at least one element.

The proposal specifies that `if_empty` is only used when the embedded resource contains no bytes. Otherwise, it has no effect.

. . .

What Happens to `prefix` and `suffix` for Empty Files?

There's one subtle detail that's worth knowing.

If the resource is **empty** and `if_empty` is used, the `if_empty` replacement is used instead of the normal embedded data. For example:

```
constexpr unsigned char data[] = {  
    #embed "empty.bin"  
    prefix(1,)  
    suffix(2,)  
    if_empty(0)  
};
```

If `empty.bin` contains no data, the generated sequence is based on the `if_empty` replacement rather than an empty resource. This behavior is defined by the proposal so empty resources can still produce meaningful initializers.

The important takeaway is that you don't need to write special handling for empty resources yourself.

. . .

Checking for Resources with `__has_embed`

So far, we've assumed the file always exists.

```
constexpr unsigned char logo[] = {  
    #embed "logo.png"  
};
```

But what happens if it doesn't? The compiler will fail because it can't find the resource. Sometimes that's exactly what you want. If your application requires a particular file to build, a compilation error is the right outcome.

However, there are situations where a resource is optional. Maybe you're building different editions of your application. Maybe some platforms include a particular firmware image while others don't. Or perhaps you're writing a library that can take advantage of an embedded resource when it's available but should still compile without it.

That's where `__has_embed` comes in.

`__has_embed` is a new preprocessor operator introduced alongside `#embed`.

It allows you to check whether a resource can be embedded before actually embedding it.

A simple example looks like this:

```
#if __has_embed("logo.png")

constexpr unsigned char logo[] = {
    #embed "logo.png"
};

#endif
```

If the resource exists and can be embedded, the code is compiled.

Otherwise, it's skipped.

This is very similar to how many developers already use `__has_include`.

If you've used this before:

```
#if __has_include(<filesystem>)
#include <filesystem>
#endif
```

then `__has_embed` should feel immediately familiar.

Instead of asking:

"Can I include this header?"

you're asking:

"Can I embed this resource?"

Here is A Practical Example:

Suppose your application can optionally display a splash screen. Some product editions include it. Others don't.

Without `__has_embed`, you might need different build scripts or generated configuration files.

With `__has_embed`, everything stays in one place.

```
#if __has_embed("splash.png")

constexpr unsigned char splash[] = {
    #embed "splash.png"
};

#endif
```

If `splash.png` isn't present, the program still compiles. No build errors. No special build configuration.

• • •

`__has_embed` **Providing a Fallback**

A common pattern is to provide an embedded resource when available and fall back to another implementation otherwise.

For example:

```
#if __has_embed("certificate.pem")

constexpr char certificate[] = {
    #embed "certificate.pem"
    suffix('\0')
};

#else

constexpr char certificate[] =
    "Default Certificate";

#endif
```

Your code now works in both situations. If the file exists, it's embedded. Otherwise, the fallback version is used.

• • •

What Does `__has_embed` Return?

Unlike many preprocessor operators that simply evaluate to `0` or `1`, `__has_embed` can report different outcomes.

The standard defines three possible results:

- The resource exists and can be embedded.
- The resource exists but is empty.
- The resource cannot be embedded (for example, it doesn't exist or isn't supported). The proposal exposes these as predefined result values that can be tested in preprocessor conditions.

Most developers won't need to distinguish between all three cases. A simple existence check is usually enough.

But if you're writing portable libraries or build infrastructure, the extra information can be useful.

• • •

Checking for an Empty Resource

Suppose your project generates a resource automatically. Sometimes that file may legitimately be empty. You can detect that case separately.

```
#if __has_embed("generated.bin") == __STDC_EMBED_EMPTY__  
  
    // Handle empty resource  
  
#endif
```

Likewise, you can explicitly check whether embedding is supported.

```
#if __has_embed("generated.bin") == __STDC__EMBED_NOT_FOUND__  
  
// Resource unavailable  
  
#endif
```

These predefined constants make your intent much clearer than testing arbitrary numeric values.

• • •

Why Not Just Try `#embed` ?

You might wonder:

Why not simply write the `#embed` and let the compiler complain if the file doesn't exist?

For many applications, that's exactly the right approach.

If the file is required, a compile-time error is helpful because it immediately tells you something is wrong.

`__has_embed` is most useful when the resource is **optional**.

Examples include:

- Optional application branding
- Platform-specific firmware
- Debug-only assets
- Sample data
- Automatically generated resources
- Library resources that may or may not be present

If the program can't function without the resource, simply using `#embed` is often the better choice.

• • •

Comparing `__has_include` and `__has_embed`

Many C++ developers already know `__has_include`. The two operators solve similar problems. If you understand one, the other feels very natural.

Feature	<code>__has_include</code>	<code>__has_embed</code>
Checks for	Header files	Embeddable resources
Used with	<code>#include</code>	<code>#embed</code>
Evaluated during	Preprocessing	Preprocessing
Typical use	Optional libraries	Optional embedded files

• • •

When Should You Use `__has_embed` ?

Most projects probably won't need it every day.
But it's a great fit when:

- Resources are optional.
- Different platforms ship different assets.
- Your build generates files that may not always exist.
- You're writing a reusable library.
- You want a clean fallback instead of a compilation error.

Otherwise, simply using `#embed` directly keeps the code simpler.

• • •

When Should You Use `#embed` ?

Like any feature, `#embed` isn't the answer to every problem.
It's best suited for resources that are part of the application itself.
Good candidates include:

- Icons
- Logos
- Fonts
- Shaders
- SSL certificates
- Firmware
- SQL scripts
- HTML templates
- Built-in configuration files
- Small machine learning models

These files rarely change once the application is built. Embedding them simplifies deployment because there's only one file to distribute.

. . .

When You Probably Shouldn't Use It

On the other hand, some resources are better left on disk.
For example:

- User documents
- User-editable configuration files
- Save games
- Log files
- Large video files
- Music libraries
- Frequently updated assets

Embedding these into your executable means changing the data requires rebuilding the entire application. In those cases, loading the files at runtime is still the better approach.

• • •

Things to Keep in Mind

Although `#embed` is straightforward, there are a few things worth remembering.

The embedded data becomes part of your executable.

If you embed a 50 MB file, your executable grows by roughly 50 MB. That's exactly what you asked the compiler to do. For small resources, this is usually not a concern. For very large assets, it may not be the right trade-off.

The file is read during compilation

Changing the resource means recompiling the source file that uses `#embed`. This is no different from modifying any other source code.

`#embed` Doesn't Compress Anything

The compiler embeds the raw bytes from the file.

If you need compression, you'll need to compress the file yourself before embedding it and decompress it at runtime.

It's Completely Portable

Perhaps the biggest advantage of `#embed` isn't that it embeds files. We've been doing that for decades. The real benefit is that **there is finally one standard way to do it**. Instead of relying on:

- `xxd`
- `objcopy`
- Windows resource files
- linker scripts
- custom Python scripts
- generated headers

every C++26 compiler understands the same directive. That makes build systems simpler and code more portable.

. . .

Before vs After

Before C++26	With C++26
Convert binary files into C++ arrays	Use <code>#embed</code>
Maintain generated headers	Keep the original resource
Custom build scripts	Standard language feature
Linker-specific solutions	Portable C++
Multiple deployment files	Optionally ship a single executable

The feature doesn’t introduce new capabilities that were impossible before. Instead, it standardizes something developers have been doing for years using a collection of different tools and platform-specific techniques.

. . .

Final Thoughts

`#embed` is one of those features that solves a long-standing problem with a surprisingly simple solution. Instead of relying on custom tools and platform-specific techniques, C++26 gives us a standard way to bundle resources directly into our programs.

It’s a small feature, but one that many C++ developers will likely find themselves using regularly.

Sometimes the best language features aren’t the ones that let us do something entirely new. They’re the ones that replace years of workarounds with a single, clean solution.

In the next part, I’ll look at what `#embed` means for binary size, startup cost, and when `mmap` is still the better choice.